



INDEX

Name... Vedant Kumar Enrollment No... 35911802824

Subject... P.S.L.P. Department... ECE

S No	Experiment Name	Date	Marks (32)				Faculty Sign
			Performance (14)	File (10)	Attendance (8)	Total (32)	
1.	Perform matrix operations using loop & conditional statements	20/1/26	12	10	8	28	Δ
2	Fitting of binomial distribution after computing mean and variance	10/2/26	11	8	8	27	Δ
3	Fitting of Poisson distribution	17/2/26	11	8	8	27	Δ
4	Fitting of Normal distribution.	17/3/26	11	7	8	26	Δ
5	Fitting of Linear Regression line	24/3/26	11	7	8	26	Δ
6	Fitting of MLR curve through data sets.	24/3/26	12	7	8	27	Δ
7	Solve a transportation problem of 3 variables	7/4/26	12	7	8	27	Δ
8	Solve an assignment problem of 3 variables	7/4/26	12	7	8	27	Δ
9	Solve a transportation problem (least cost method)	7/4/26	12	7	8	27	Δ
10	Solve a LP of 3 variables using simplex method	7/4/26	12	7	8	27	Δ

Topic.....

Date.....

Exp-1

To perform Matrix addition, subtraction and multiplication using loop and if conditional statements

MATLAB name stands for MATRIX Laboratory. MATLAB is a high-level software used for numerical computation, signal processing, graphics and signal analysis. It provides 2-D and 3-D graphics, supports computational like linear algebra, interpolation, numerical integration, and ODE solution, and can interface with C and FORTRAN. Additional features are available through toolboxes such as signal processing, Image processing, and Control system.

In DSP, MATLAB represents signals as sequence of numbers and processes them to extract useful information. MATLAB uses m-files and mex-files and works as an interactive language. So no compilation is required.

MATLAB has 3 main windows:

- Command Window
- Graphics Window
- Edit Window

Input Subportion

enter rows - 3

enter columns - 3

enter matrix A - [3 4 1 ; 4 2 1 ; 2 6 1]

enter matrix B [4 6 2 ; 3 2 1 ; 1 2 3]

$\begin{bmatrix} 1 & -1 & -1 \\ 1 & 0 & 0 \\ 1 & 1 & -2 \end{bmatrix}$

Output Multiplication

No. of rows - 3

No. of columns - 3

No. of rows - 3

No. of columns - 3

enter matrix A - [1 1 1 ; 1 1 1 ; 1 1 1]

enter matrix B - [1 1 1 ; 1 1 1 ; 1 1 1]

C =

3 3 3

3 3 3

3 3 3

Topic.....Date.....

Multiplication

```

r1 = input("No. of rows");
c1 = input("No. of columns");
r2 = input("No. of rows");
c2 = input("No. of columns");
A = input("Enter matrix A");
B = input("Enter matrix B");
C = zeros(r1, c2);
if r1 == c1

```

```

    for i = 1:r1

```

```

        for j = 1:c2

```

```

            x = 0;

```

```

            for k = 1:r2

```

```

                x = x + (A(i, k) * B(k, j));

```

```

            C(i, j) = x;

```

```

        end

```

```

    end

```

```

end

```

```

end

```

∴ Hence, matrix addition, subtraction & multiplication was successfully performed.

30

20/1/2026

Teacher's Sign.....

Exp-2

Fitting of binomial distribution after computing mean and variance.

The In probability theory and statistics, the binomial distribution is the discrete probability distribution that gives only 2 possible results in an exp. either Success or Failure.

This distribution is also called a binomial probability distribution.

There are 2 parameters. n and p used here in a binomial distribution. The variance ' n ' states the no. of times the experiment runs and the variable ' p ' tells the probability of any one outcome.

The binomial distribution formula is for any random variable x , given by:

$$P(x; n, p) = {}^n C_x p^x (1-p)^{n-x}$$

or

$$P(x; n, p) = {}^n C_x p^x (q)^{n-x}$$

where,

n = the no. of experiments

$x = 0, 1, 2, 3, 4, \dots$

p = Probability of Success in a single exp.

q = prob. of Failure in " exp. = $1-p$

The binomial distribution formula can ~~be~~ also be written in the form of n -Bernoulli trials, where, $nC_x = n! / x!(n-x)!$. Hence;

$$P(x:n,p) = n! / [x!(n-x)!] p^x (1-p)^{n-x}$$

For a binomial distribution, the mean, variance and standard deviation for the given n of success are represented using the formulae.

• Code

```
function [mean, variance, SD] = vecbinom(n,p)
n=400;
p=0.1;
mean = n*p;
variance = n*p*(1-p);
SD = sqrt(variance);
x=0:n;
y = binopdf(x,n,p);
figure;
bar(x,y,1)
xlabel('observation');
ylabel('probability');
end.
```

- The following has been observed carefully in MATLAB.

So

EXPERIMENT-2

```
1 function[mean,variance,SD]=example2(n,p)
2 n= input('The No. of Experiments: ');
3 p=input('Probability of Success in a Signal Experiment: ');
4 % Mean,variance and standard deviation is computed
5 mean=n*p;
6 variance=n*p*(1-p);
7 SD=sqrt(variance);
8 %pdf
9 x=0:n;
10 y=binopdf(x,n,p);
11 %figure
12 bar(x,y,1);
13 xlabel('Observation');
14 ylabel('Probability');
15
```

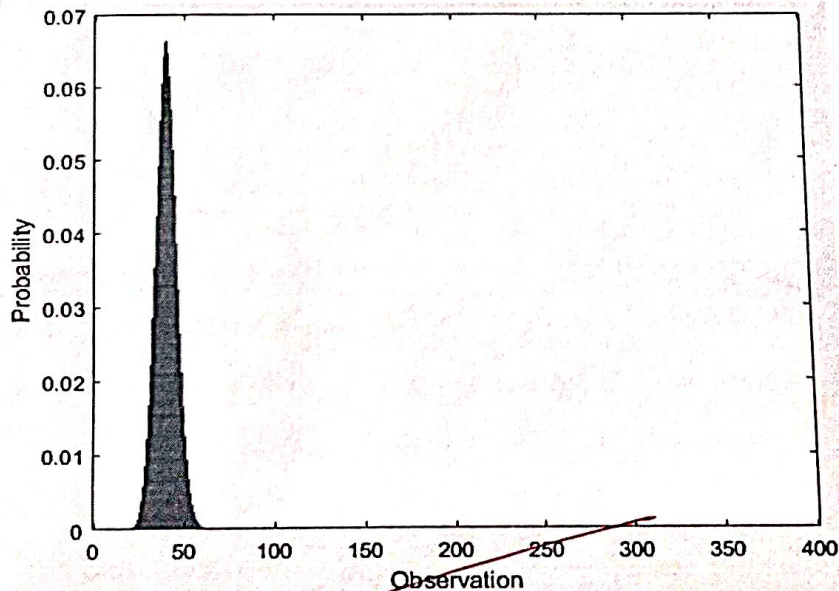
Command Window

```
>> example21
The No. of Experiments: 400
Probability of Success in a Signal Experiment: 0.1
```

```
ans =
```

```
40
```

f_x >>



Topic.....Date.....

Case Study - 1

Calculate the prob. of getting atleast 2 heads in 10 tosses.

MATLAB CODE:

```
clc;
clear;
n=10;
p=0.5;
x=0:n;
y=binopdf(x,n,p);
```

```
figure;
stem(x,y);
xlabel('No. of Heads');
ylabel('Binomial Probability');
title('Binomial Distribution (n=10, p=0.5)');
grid on;
```

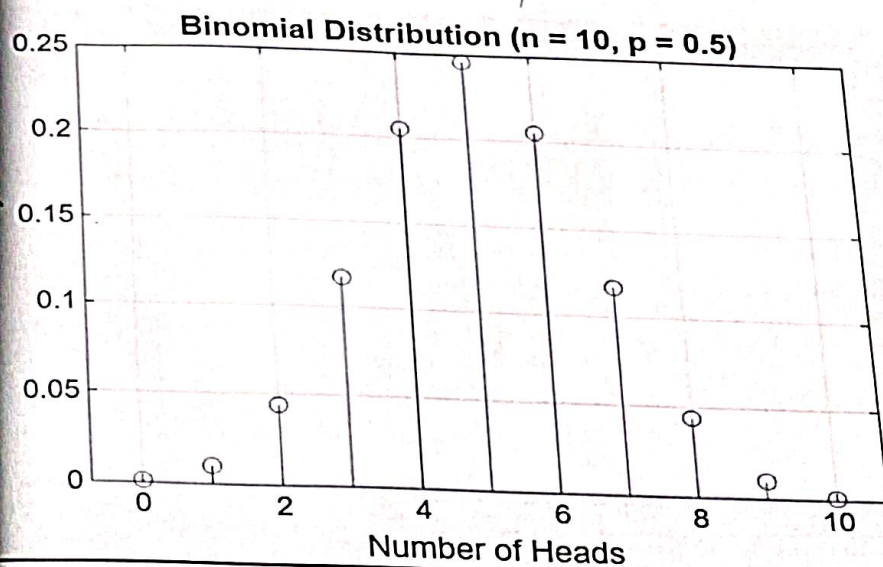
If a coin is tossed 5 times using binomial distribⁿ find the prob. of exactly 2 heads.

MATLAB CODE

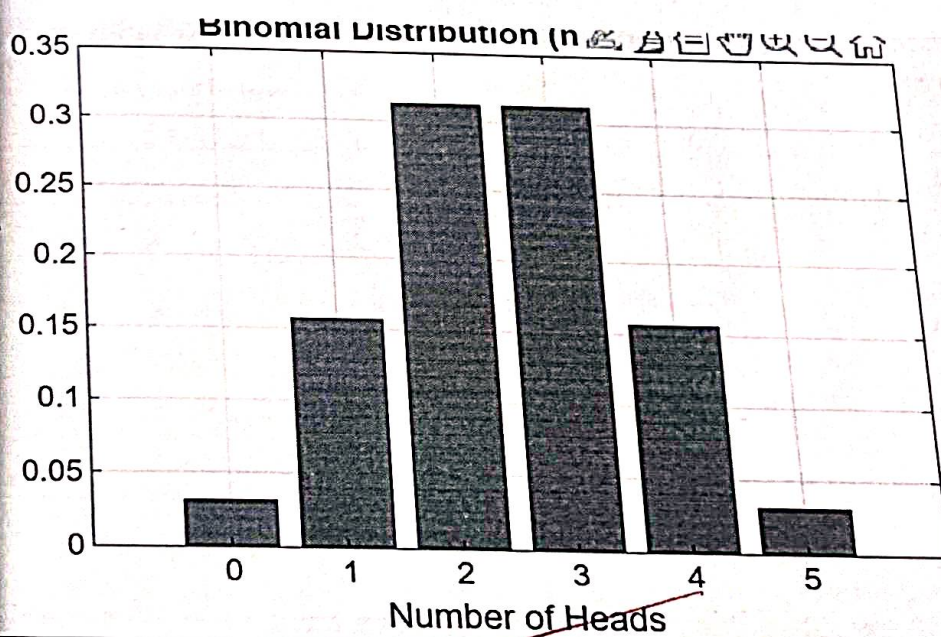
```
clc;
clear;
n=5;
p=0.5;
x=0:n;
y=binopdf(x,n,p);
figure;
stem(x,y);
```

Teacher's Sign.....

ASE STUDY-1

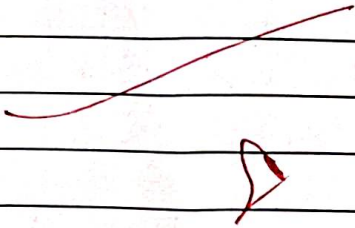


ASE STUDY-2



Topic.....Date.....

x label ('No. of Heads');
y label ('Probability');
Title ('Binomial Distribution ($n=5, p=0.5$)');
grid on;



Teacher's Sign.....

Exp 3

Plotting of Poisson distribution for given value of lambda
inputted with MATLAB software.

Poisson distribⁿ is a discrete prob. distribⁿ that represents the no. of times an event occurs in interval.

It is used when events occur independently and the avg. rate of occurrence is constant.

The prob^{ab} of Poisson distribution.

$$f(x) = P(X=x) = \frac{e^{-\lambda} \lambda^x}{x!}$$

$x \rightarrow$ Poisson random variable

$\lambda =$ avg rate (mean)

e

$$E(X) = \lambda$$

$$V(X) = \lambda$$

mean and variance are equal

$$f(x) = \frac{1}{\sigma \sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Code.

de

clear

$\lambda = 0.100;$

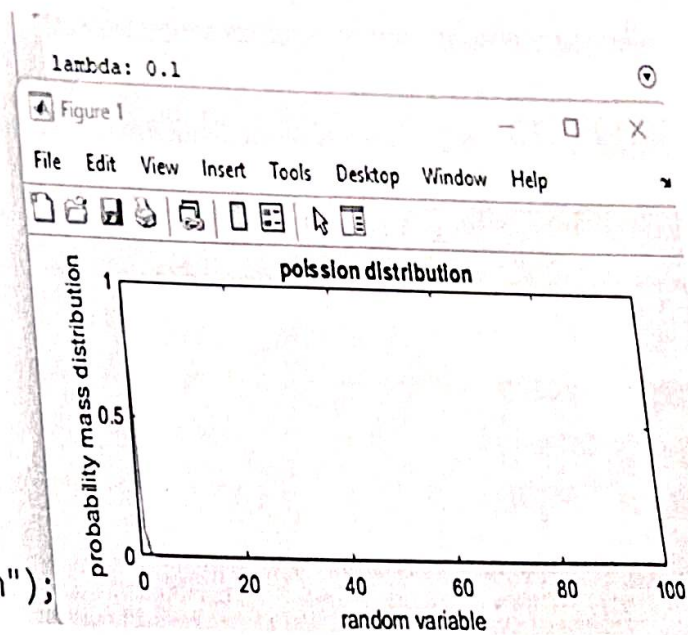
lam = input('lambda');

f = exp(-lam) * (lam.^x) ./ factorial(x);

```

clc;
clear;
close all;
x=0:100;
lam=input('lambda: ');
fx=exp(-lam)*(lam.^x)./factorial(x);
mean=sum(x.*fx);
var=sum(((x-mean).^2).*fx);
plot(x,fx);
title("poission distribution");
xlabel('random variable');
ylabel("probability mass distribution");

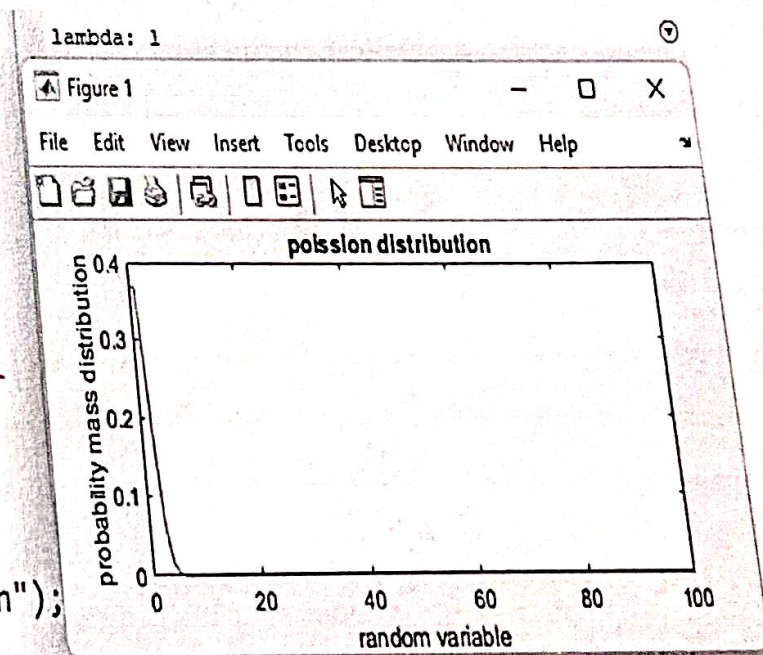
```



```

clc;
clear;
close all;
x=0:100;
lam=input('lambda: ');
fx=exp(-lam)*(lam.^x)./factorial(x);
mean=sum(x.*fx);
var=sum(((x-mean).^2).*fx);
plot(x,fx);
title("poission distribution");
xlabel('random variable');
ylabel("probability mass distribution");

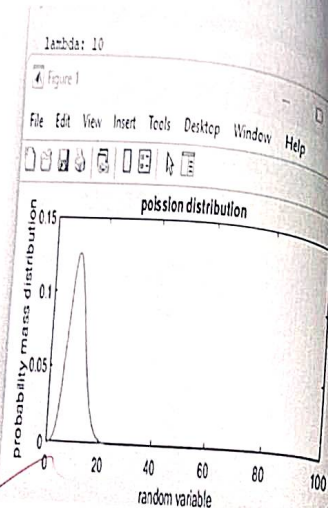
```



```

clc;
clear;
close all;
x=0:100;
lam=input('lambda: ');
fx=exp(-lam)*(lam.^x)./factorial(x);
mean=sum(x.*fx);
var=sum(((x-mean).^2).*fx);
plot(x,fx);
title("poission distribution");
xlabel('random variable');
ylabel("probability mass distribution");

```



TOPIC.....

DATE.....

$mean = \sum(x \cdot f(x))$
 $var = \sum((x - mean)^2 \cdot f(x))$

mean =

var =

plot(x,fx)

title('Poission Distribution');

xlabel('random variable');

ylabel('f(x)');

Result

The Poisson distribⁿ was successfully plotted for different values of lambda (λ). It was verified that mean and variance are equal to λ and the distribution approaches normal shape for large λ.

TOPIC.....

DATE.....

Exp-4

Aim

Filtering of Normal distribution given parameters

Theory

The Normal distribution is defined by the probability density function for a continuous random variable in a system.

$$f(x, \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-x^2/2\sigma^2}$$

 $x \rightarrow$ variable $\mu \rightarrow$ mean $\sigma \rightarrow$ standard variable

Normal Distribution

A normal distribution curve is a smooth, bell-shaped curve used for continuous variables. Values can theoretically range from $-\infty$ to $+\infty$. It helps find the probability of a value within a certain range.

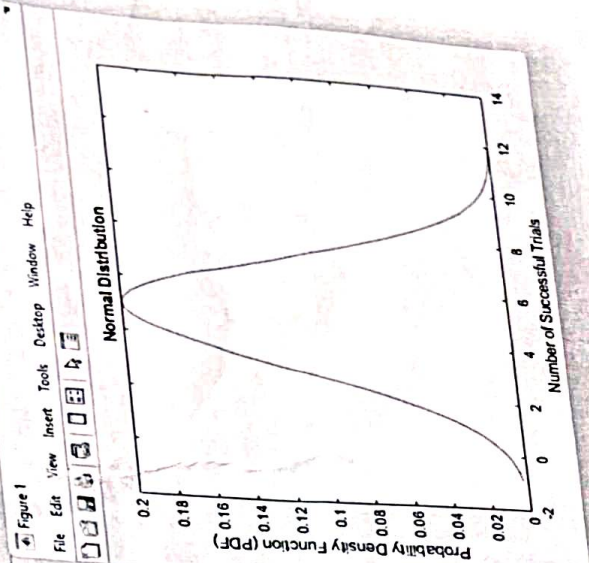
Program Code

```
clear;
clear all;
close all;
n=input('n:');
sigma=input('Sigma:');
mu=input('mu:');
```

```

1 clc;
2 clear all;
3 close all;
4 n=input('n:');
5 sigma=input('sigma:');
6 mu=input('mu:');
7 data=normrnd(5,2,[n,1]);
8 pdf=@(x,mu,sigma)(1/(sigma*sqrt(2*pi)))*(exp(-(x-mu).^2/(2*sigma^2)));
9 estimated_mu=mean(data);
10 estimated_sigma=std(data);
11 x=linspace(min(data),max(data),100);
12 plot(x,pdf(x,mu,sigma));
13 xlabel('Number of Successful Trials')
14 ylabel('Probability Density Function (PDF)')
15 title('Normal Distribution')
16
17

```



TOPIC.....

DATE.....

```
data = normrnd(5, 2, [m, 1]);  
pdf = @(x, mu, sigma) (1 / (sigma * sqrt(2 * pi))) * exp(-(x - mu)^2 / (2 * sigma^2));  
estimated_mu = mean(data);  
estimated_sigma = std(data);  
x = linspace(min(data), max(data), cr);  
plot(x, pdf(x, mu, sigma),  
xlabel('No. of successful trials'),  
ylabel('Prob. Density, fraction PDF'),  
title('Normal distribution'))
```

Result:- The Normal Distribution is studied and verified.



exp 5

Aim

Fitting of linear regression line through given data set and testing of goodness of fit using measures.

Theory

Linear regression: predicts the relationship between variables by assuming a linear connection between the independent, dependent variables.

Applied in various domains like economics and finance, suitable for binary classification problems.

Linear regression shows the linear relationship between the input variable and output variable. i.e. input called linear regression and linear regression is simple linear regression.

Code

```
def:
def calc:
x=(0:0.1:5);
y=2*x+5+10*(rand(size(x))-0.5);
scatter(x,y,'b','filled');
hold on;
b=x/y;
y=calc2=b.*x;
X=[ones(size(x)),x];
B=X\b;
plot(x,y,'calc2=b', 'line with', 2);
Rsq1=sum((y-y-calc2).^2)/(length(x));
Rsq2=sum((y-calc2).^2)/length(x);
legend('Random pts', 'Estimate w/o bias', 'Estimate with bias');
```

TOPIC.....

DATE.....

`plot(f('Non square error (without bias)', f'n, R_val, 's');
set(gcf, 'color', 'u');`

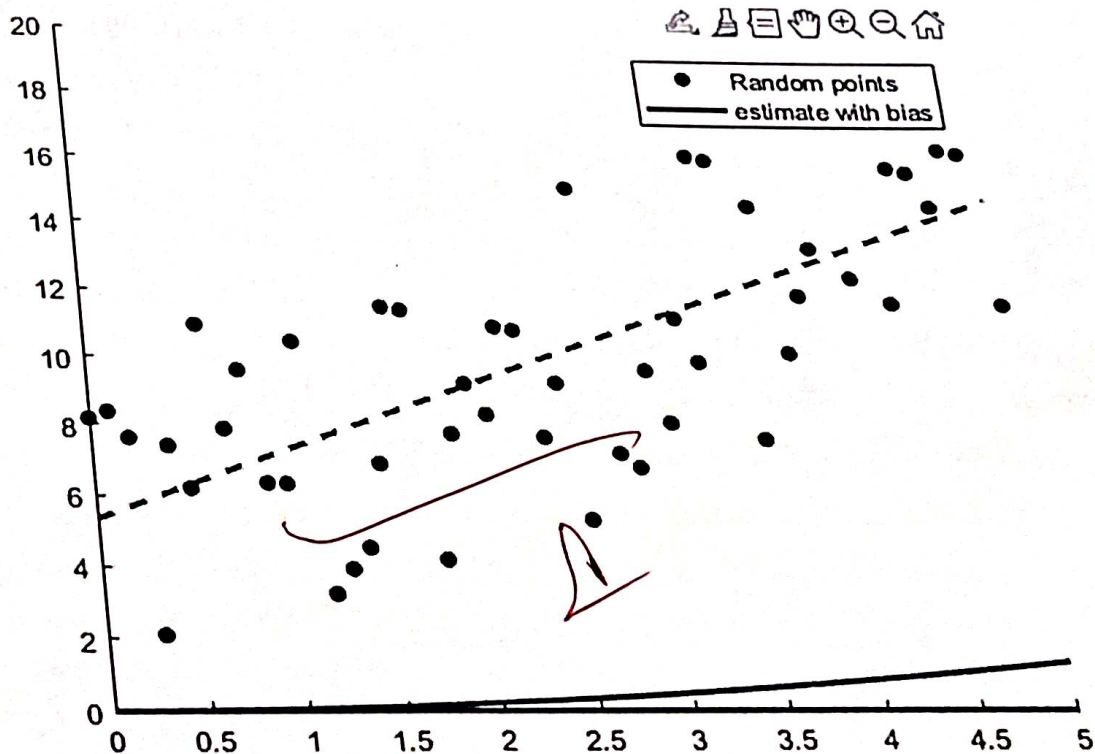
The regression line was fitted and showed a fit based on mean error.

Experiment 5 vedansh 35911502824

```
clc;
close all;
x=(0:0.1:5)';
y=2*x+5+10*(rand(size(x))-0.5);
scatter(x,y,'k','filled');
hold on;
b=x/y;
y_calc1=b.*x;
plot(x,y_calc1,'k','LineWidth',2);
X=[ones(size(x)),x];
B=X\y;
y_calc2=B(1)+B(2).*x;
plot(x,y_calc2,'--k','LineWidth',2);
R_sq1=sum((y-y_calc2).^1)/length(x);
R_sq2=sum((y-y_calc2).^2)/length(x);
legend('Random points','estimate with bias');
fprintf('Mean square error(without bias):%f\n',R_sq1,'r');
fprintf('Mean square error(with bias):%f\n',R_sq2);
set(gcf,'color','w');
```

Figure 1

File Edit View Insert Tools Desktop Window Help



Exp-6

Aim- Fitting of Multiple Linear Regression (MLR) curve through given data set and testing of goodness of fit using mean error.

Theory ~~Multiple~~ MLR is statistical technique that uses several independent variables to predict the outcome of a response variable. The MLR is extension of ordinary least-squares regression bcoz it involves more than 1 explanatory variable.

- Formula and Calcⁿ of MLR.

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} + \dots$$

for $i = n$ observations

Linear Regression can only be used when we have 2 continuous variables - an independent variable and a dependent variable.

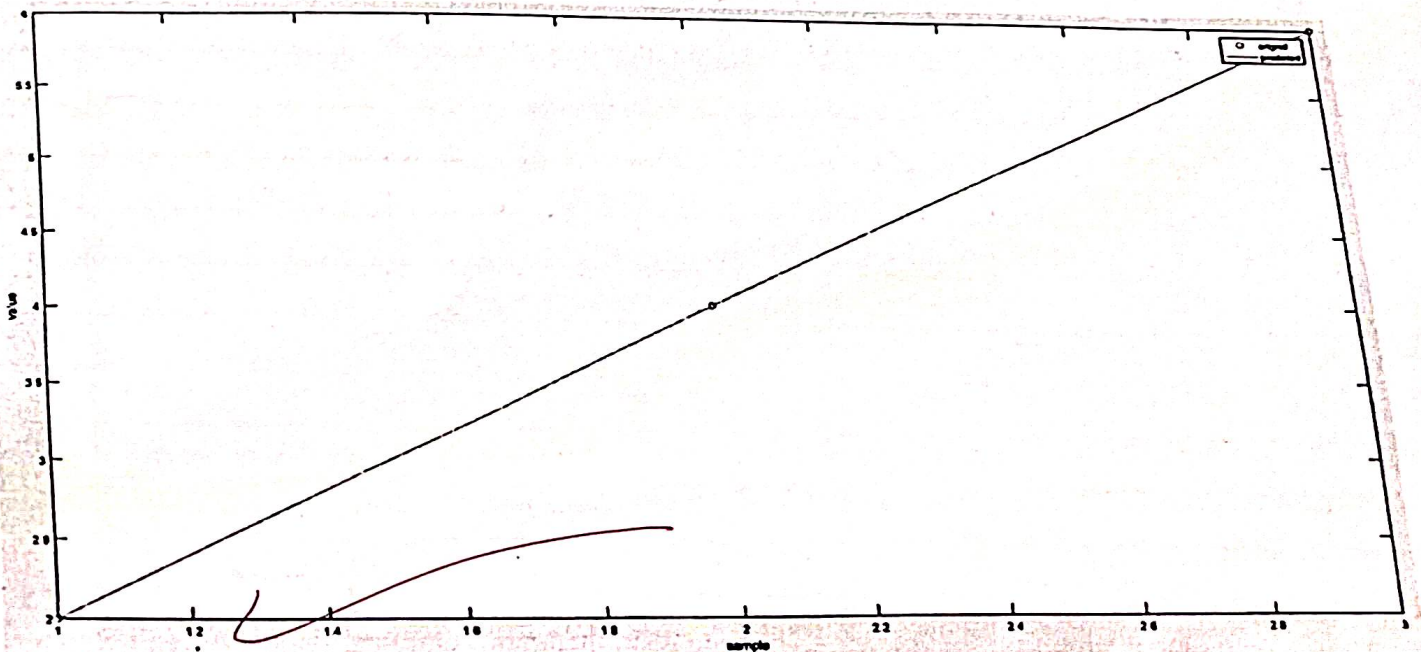
The parameter that's used to calculate the dependent variable is outcome.

Code

```
x = [1, 2, 3, 4, 5; 2, 4, 6, 8, 10; 1, 1, 1, 1, 1];
y = [2; 4; 6];
b = regress(y, x);
by_pred = x * b;
plot(y, 'o', 'Display name', 'original');
hold on;
```

EXPERIMENT-6

```
x=[1,2,3,4,5;2,4,6,8,10;1,1,1,1,1];  
y=[2;4;6];  
b=regress(y,x);  
y_pred=x*b;  
plot(y,'o','DisplayName','original');  
hold on;  
plot(y_pred,'DisplayName','predicted');  
xlabel('sample');  
ylabel('value');  
legend('show');
```



20

TOPIC.....

DATE.....

```
xlabel('sample');  
ylabel('value');  
legend('show');
```

Result- Hence, MLP is studied & graph is verified.

Q

Exp-7

Ques- Solve a transportation problem (North West corner fill cell Method) w/ 3 variables.

Ans- North - West Corner Rule is a method to find an initial basic feasible solⁿ for a transportation problem.

Steps.

- 1) Start from top-left (N-W) cell
- 2) Allocate min. of supply & demand to cell
- 3) Reduce corresponding row supply & column demand
- 4) If supply becomes 0 $\rightarrow$ move down
If demand becomes 0 $\rightarrow$ move right.
- 5) Repeat all till are satisfied.

Key pts.

- Simple & quick method.
- Give a feasible solⁿ.
- No. of allocations $\geq (m + n - 1)$.
- Used as start solⁿ for optimization.

Code

```

a = input("Enter transportation matrix:")
supply = input("Enter supply");
demand = input("Enter demand");
[m, n] = size(a);
x1 = zeros(m, n);
sum = 0;
j = 1;
i = 1;

```

MATLAB CODE:

```
x=input('Enter the Transportation Matrix: ');
supply=input('Enter Supply: ');
demand=input('Enter Demand: ');
[m,n]=size(x);
x1=zeros(m,n);
sum=0;
i=1;
j=1;
while i<=m && j<=4
    min1=min(demand(j) & supply(i));
    sum=sum+x(i,j)*min1;
    demand(j)=demand(j)-min1;
    supply(i)=supply(i)-min1;
    if demand(j)==0
        j=j+1;
    end
    if supply(i)==0
        i=i+1;
    end
end
disp(['The Transportation cost is: ',num2str(sum)]);
```

OUTPUT:

Command Window

```
Enter the Transportation Matrix: [8 6 10 9; 9 12 13 7; 14 9 16 5]
Enter Supply: [20 30 25]
Enter Demand: [10 25 25 15]
The Transportation cost is: 750
```

```
>> !
```

```
supply(i) = supply(i) - min(i);  
if demand(j) = 0  
    j = j + 1  
end  
if supply(i) = 0  
    i = i + 1;  
end  
end.  
[sumD];
```

Result Hence, the transportation cost is 7501

1

Qap-8

Aim- Solve an assignment problem of 3 variables.

Theory- The assignment prob is special case of LP used to assign or workers for n jobs, on 1-to-1 basis.

Each job is assigned exactly only one worker.
The cost of assigning worker i to job j is c_{ij} .

- Maths formulation

Minimize

$$Z = \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij}$$

subject to:

$$\sum_{i=1}^m x_{ij} = 1 \text{ for each job } j$$

Decision variable:

$$x_{ij} = \begin{cases} 1, & \text{if worker } i \text{ is assigned to job } j \\ 0, & \text{otherwise} \end{cases}$$

where, m = no. of workers =

n = no. of jobs =

~~c_{ij} = cost of assignment.~~

Key word $\rightarrow m = n$

MATLAB CODE:

```

%PSLP03.m
clc; clear;

% Input cost matrix
arr = [3 3 2 8;
       7 9 2 6;
       6 4 5 7;
       5 7 7 8];

disp('Original Cost Matrix:');
disp(arr);

[n, m] = size(arr);
orig = arr;

%% STEP 1: Row Reduction
for i = 1:n
    arr(i,:) = arr(i,:) - min(arr(i,:));
end

%% STEP 2: Column Reduction
for j = 1:m
    arr(:,j) = arr(:,j) - min(arr(:,j));
end

disp('After Row & Column Reduction:');
disp(arr);

%% STEP 3: Cover zeros and modify matrix
%PSLP03 true

temp = arr;
row_cover = false(n,1);
col_cover = false(m,1);

lines = 0;

```

```

%PSLP03.m
while true

    zero_row = sum(temp==0,2);
    zero_col = sum(temp==0,1);

    % Avoid empty min issue
    zero_row(zero_row==0) = inf;
    zero_col(zero_col==0) = inf;

    min_row = min(zero_row);
    min_col = min(zero_col);

    if isinf(min_row) && isinf(min_col)
        break;
    end

    if min_row <= min_col
        r = find(zero_row==min_row,1);
        row_cover(r) = true;
        temp(r,:) = inf;
    else
        c = find(zero_col==min_col,1);
        col_cover(c) = true;
        temp(:,c) = inf;
    end

    lines = lines + 1;
end

% If optimal condition satisfied
if lines >= n
    break;
end

```

```

%% Find minimum uncovered value
min_val = inf;

```

```

        end
    end
end
if isinf(min_val)
    break;
end

XX Update matrix
for i = 1:n
    for j = 1:m
        if ~row_cover(i) && ~col_cover(j)
            arr(i,j) = arr(i,j) - min_val;
        elseif row_cover(i) && col_cover(j)
            arr(i,j) = arr(i,j) + min_val;
        end
    end
end

disp('Final Reduced Matrix:');
disp(arr);

XX STEP 4: Optimal Assignment (SAFE)
assignment = -ones(n,1);
used_cols = false(m,1);

% First pass
for i = 1:n
    for j = 1:m
        if arr(i,j) == 0 && ~used_cols(j)
            assignment(i) = j;
            used_cols(j) = true;
            break;
        end
    end
end

% Fix unassigned rows
for i = 1:n
    if assignment(i) == -1
        for j = 1:m
            if arr(i,j) == 0
                assignment(i) = j;
                used_cols(j) = true;
                break;
            end
        end
    end
end

XX FINAL SAFETY CHECK
if any(assignment == -1)
    error('Assignment incomplete - algorithm needs refinement');
end

XX STEP 5: Total Cost
total_cost = 0;
for i = 1:n
    total_cost = total_cost + orig(i, assignment(i));
end

disp('Assignment (Row -> Column):');
disp(assignment);

disp('Total Minimum Cost:');

```

	1	2	3	4
1	1	2	3	4
2	2	3	4	5
3	3	4	5	6
4	4	5	6	7
5	5	6	7	8

	1	2	3	4
1	1	2	3	4
2	2	3	4	5
3	3	4	5	6
4	4	5	6	7
5	5	6	7	8

	1	2	3	4
1	1	2	3	4
2	2	3	4	5
3	3	4	5	6
4	4	5	6	7
5	5	6	7	8

Total Minimum Cost:
19

TOPIC.....

DATE.....

Cost Matrix (General form):

Worker	Job 1	Job 2	...	Job n
Worker 1	C_{11}	C_{12}	...	C_{1n}
W 2	C_{21}	C_{22}	...	C_{2n}
...	...	...	...	...
W m	C_{m1}	C_{m2}	...	C_{mn}

Example

	Job 1	Job 2	Job 3
W 1	15	10	9
W 2	9	15	10
W 3	10	12	8

Result Hence, ~~total cost is 13~~, total cost is 13.

1

Exp-9

Ques. Solve a transportation problem (least corner method).

Theory: The Least Cost Method is another method used to obtain the initial feasible solⁿ for the transportation problem.

The Least Cost Method is considered to produce more optimal than the North-West corner because it considers the shipping cost with the extreme left corner, irrespective of shipping cost.

Following are steps to solve the LCM.

Step 1) Balance the problem.

Step 2) Select the lowest cost from entire matrix and allocate min. of supply or demand.

Step 3) Remove the row or column whose supply or demand is fulfilled.

Step 4) Repeat the procedure until all the allocations are done.

Code

clc;

clear all;

close all;

A=[1, 2 3 4; 5 3 2 0; 0 2 2 1];

MATLAB CODE:

```

clc;
close all;
clear all;
A=[1 2 3 4;4 3 2 0; 0 2 2 1];
supply=[6 8 10];
demand=[4 6 8 6];
y=0;
while (~isempty(supply) && ~isempty(demand))
    M=min(A);
    N=min(M);
    i=0;
    j=0;
    [i,j]=find(M);
    X=min(supply(i),demand(j));
    y=y+X*N;
    if (X==supply(i))
        A(i,:)=[];
        supply(i)=[];
        demand(j)=demand(j)-X;
    end
    if (X==demand(j))
        A(:,j)=[];
        demand(j)=[];
        supply(i)=supply(i)-X;
    end
end
disp('The last cost is of the given input matrix is:')
disp(y)

```

OUTPUT:

Command Window

The last cost is of the given input matrix is:
28

A

Expt-10

Ques- Solve LPP of 3 variables using simplex method.

Ans- Simplex method, standard technique in linear programming for solving an optimization problem, typically of involving a funcⁿ and several set constraints expressed as inequalities.

The simplex method is an approach to solving LPP models by hand using loop variables; pivot variables, also means to finish the optimal solⁿ of an optimization problem.

- Standard form.
- Introducing slack variable.
- Creating the tables.
- Pivot variables.
- Creating a new optimality.
- Identifying optimal values.

Result- The current BFs is optimal is:

x_1	x_2	x_3	S_1	S_2	S_3	Sol
0	0	2.4	0.2	0.8	0	11
1	0	0.8	0.4	0.1	0	4

</

```

for i = 1:length(col)
    if col(i) > 0
        ratio(i) = sol(i) / col(i);
    else
        ratio(i) = inf;
    end
end
[min_ratio, pivotRow] = min(ratio);
fprintf('Pivot row = %d\n', pivotRow);
fprintf('leaving variable = %d\n', basic_var(pivotRow));
end
% update basic variable
basic_var(pivotRow) = pivotCol;
disp('New basic variables:');
disp(basic_var);
% pivot operation
pivot_key = A(pivotRow, pivotCol);
A(pivotRow,:) = A(pivotRow,:) / pivot_key;
for i = 1:size(A,1)
    if i ~= pivotRow
        A(i,:) = A(i,:) - A(i,pivotCol) * A(pivotRow,:);
    end
end
% update Zj-Cj
ZjCj = cost(basic_var)*A - cost;
% display table
t = [ZjCj; A];
table = array2table(t);
table.Properties.VariableNames = {'x1','x2','x3','s1','s2','s3','sol'};
disp(table);
% BFS calculation
BFS = zeros(1,size(A,2));
BFS(basic_var) = A(:,end);
BFS(end) = sum(BFS .* cost);
current_BFS = array2table(BFS);
current_BFS.Properties.VariableNames = {'x1','x2','x3','s1','s2','s3','sol'};
disp(current_BFS);
else
    RWI = false;
    fprintf('-----\n');
    fprintf('The current BFS is OPTIMAL\n');
    fprintf('-----\n');
end
end

```

OUTPUT:

Command Window

	x1	x2	x3	s1	s2	s3	sol
	—	—	—	—	—	—	—
1		-3	2	0	0	0	0
3		-1	2	1	0	0	7
-2		4	0	0	1	0	12
-4		3	8	0	0	1	10

The current BFS is not Optimal

===== Next Iteration =====

Old basic variables:

4 5 6

Minimum element in $Z_j - C_j = -3.000000$ at column 2

Entering variable = x2

Pivot row = 2

Leaving variable = 5

New basic variables:

4 2 6

	x1	x2	x3	s1	s2	s3	sol
	—	—	—	—	—	—	—
-0.5		0	2	0	0.75	0	9
2.5		0	2	1	0.25	0	10
-0.5		1	0	0	0.25	0	3
-2.5		0	8	0	-0.75	1	1

	x1	x2	x3	s1	s2	s3	sol
	—	—	—	—	—	—	—
0		3	0	10	0	1	9

The current BFS is not Optimal

===== Next Iteration =====

Old basic variables:

4 2 6

Minimum element in $Z_j - C_j = -0.500000$ at column 1

Entering variable = x1

Pivot row = 1

Leaving variable = 4

New basic variables:

1 2 6

	x1	x2	x3	s1	s2	s3	sol
	—	—	—	—	—	—	—
0		0	2.4	0.2	0.8	0	11
1		0	0.8	0.4	0.1	0	4



TOPIC.....

supply = [6 8 10];
demand = [4 6 8 6];

```

g = 0;
while (~isempty(supply) & ~isempty(demand))
    M = min(A);
    N = min(M);
    i = 0;
    j = 0;
    [i, j] = find(A == N);
    x = min(supply(i(1)), demand(j(1)));
    g = g + x * N;
    if (x == supply(i(1)))
        supply(i(1)) = [];
        demand(j(1)) = demand(j(1)) - x;
    end
end

```

```

if (x == demand(j(1)))
    A(i, j(1)) = [];
    demand(j(1)) = [];
    supply(i(1)) = supply(i(1)) - x;
end
end

```

disp('The last cost is of given i/p matrix is:');
disp(g)

Result The last cost is of the given i/p matrix is 28928

